Claude Code
State Machine Analysis Report

A visual guide to the 7 state machines that drive Claude Code

March 31, 2026

Modeled with Orca | Verified structurally | Visualized with Mermaid.js

What Is Orca?

Orca is a state machine framework that provides a structured, verifiable way to model, analyze, and generate code for finite state machines. Given a machine description in .orca.md format, Orca can verify structural properties, generate XState v5 TypeScript code, and produce Mermaid.js diagrams for visualization. Source: github.com/nomyx-io/orca-lang

Why Use Orca for Analysis?

Software systems often contain implicit state machines—code that manages transitions between named states without a formal model. Orca lets us reverse-engineer these patterns into verifiable models and check five structural properties:

- **Reachability:** Can every state be reached from the initial state? Unreachable states indicate dead code or missing transitions.
- **Deadlock freedom:** Can the machine always make progress from non-final states? Deadlocked states indicate missing event handlers.
- **Event completeness:** Does every state explicitly handle (or ignore) every event? Unhandled events can cause silent failures.
- **Guard determinism:** When multiple guarded transitions exist for the same state+event, are the guards mutually exclusive? Overlapping guards create ambiguous behavior.
- **Guard exhaustiveness:** Do the guards for a state+event combination cover all possible context values? Gaps mean some inputs have no defined behavior.

The Orca Workflow

- **Generate:** Create an .orca.md model from a natural language specification or by reverse-engineering source code.
- **Verify:** Run structural checks (reachability, deadlocks, completeness, guard analysis) to find issues in the model.
- **Iterate:** Fix issues found by verification—add missing transitions, ignore directives, or guard clauses—until the model passes.
- **Compile:** Generate XState v5 TypeScript code or Mermaid diagrams from the verified model.
- **Visualize:** Render the machine as an interactive state diagram for documentation and review.

In this analysis, we applied this workflow to 7 state machines discovered in the Claude Code codebase. Each machine was modeled in Orca, verified, and visualized.

State Machine Overview

Claude Code contains 7 distinct state machines spanning authentication, networking, input handling, voice, feedback, session management, and worktree lifecycle. The table below summarizes each machine's complexity and verification status.

Machine	States	Events	Trans.	Complexity	Verification	Impl.
Session Lifecycle	3	5	5	Low	VALID	Implicit
MCP Server Connection	5	9	13	Medium	VALID	Explicit
OAuth Auth Flow	8	12	18	Med-High	VALID	Explicit
Voice Input	3	7	7	Low-Med	VALID	Implicit
Feedback Survey	6	9	11	Medium	VALID	Implicit
Worktree Exit Dialog	5	5	6	Low-Med	VALID*	Implicit
Vim Command Parser	12	2	51	High	VALID	Explicit

** 1 guard exhaustiveness warning (see Appendix A.6)*

How to Read the Diagrams

- Rounded rectangles represent states. The filled circle marks the initial state.
- Arrows represent transitions, labeled with the event or condition that triggers them.
- States with a double border are final states where the machine’s work is complete.
- Self-loops (arrows back to the same state) represent events that cause side effects without changing state.

1. Session Lifecycle

The Session Lifecycle is the highest-level state machine in Claude Code. It tracks whether the application is waiting for user input, processing a request, or blocked on a permission decision.

How It Works

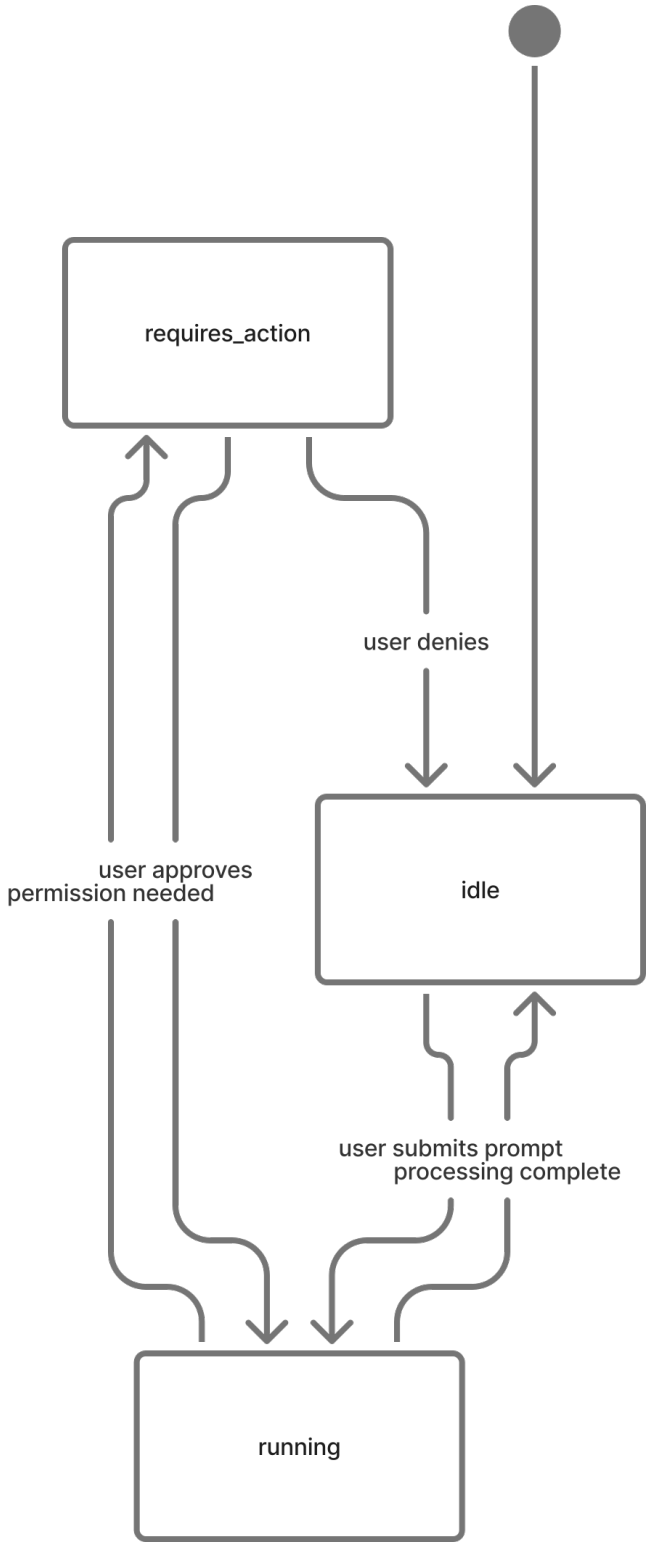
When a user submits a prompt, the session transitions from **idle** to **running**. If Claude needs to execute a tool that requires permission (file edits, shell commands), the session moves to **requires_action** and waits. Once the user approves or denies, the session returns to **running** or **idle** respectively. When processing completes, the session returns to **idle**.

Property	Value
Source	src/utils/sessionState.ts
Implementation	Implicit (string type + notification function)
States	idle, running, requires_action
Verification	VALID (0 errors)

Design Notes

This is a direct-assignment state machine—there is no event dispatch or transition table. The code calls `notifySessionStateChanged()` with the new state, which broadcasts to SDK consumers, CCR sidebar, and push notification channels.

Figure 1: Session Lifecycle State Diagram



2. MCP Server Connection

Claude Code connects to external tools via the Model Context Protocol (MCP). Each MCP server connection has its own lifecycle, from initial connection attempt through authentication, failure, and recovery.

How It Works

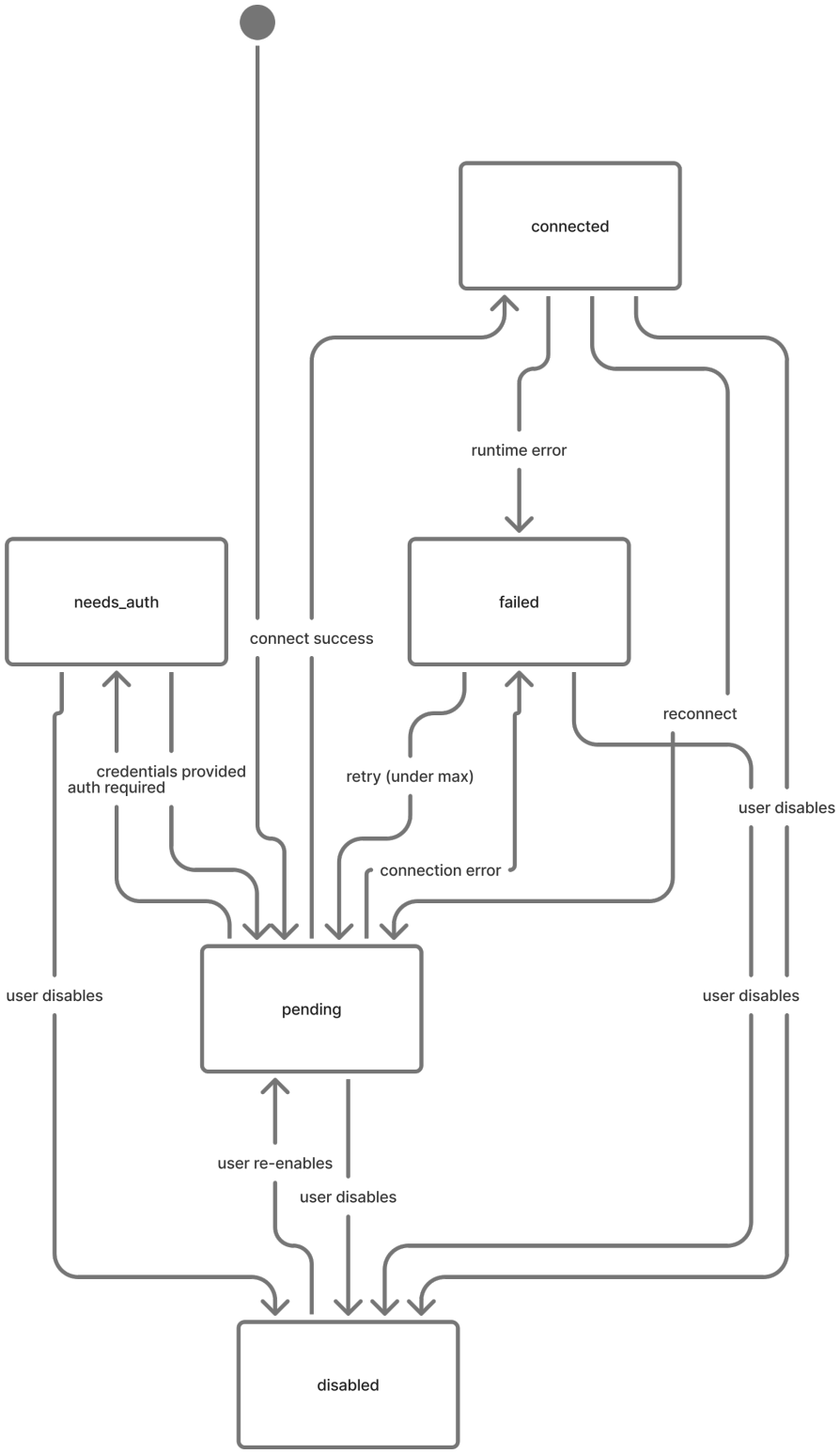
When Claude Code starts, each configured MCP server begins in the **pending** state, attempting to connect. On success it becomes **connected**; on failure it moves to **failed** with optional retry logic. If the server requires OAuth or other credentials, it enters **needs_auth** until the user provides them. Any server can be manually **disabled**. Connected servers that encounter runtime errors fall back to **failed** and may reconnect.

Property	Value
Source	src/services/mcp/types.ts:180–226
Implementation	Explicit (TypeScript discriminated union)
States	pending, connected, needs_auth, failed, disabled
Verification	VALID (0 errors)

Design Notes

Uses TypeScript's discriminated union pattern—each state variant (`ConnectedMCPServer`, `FailedMCPServer`, etc.) carries different data. The `type` field acts as the discriminant.

Figure 2: MCP Server Connection State Diagram



3. OAuth Authentication Flow

When a user first runs Claude Code, they need to authenticate. The OAuth flow manages this multi-step process across four providers, handling browser redirects, API key creation, errors, and automatic retries.

How It Works

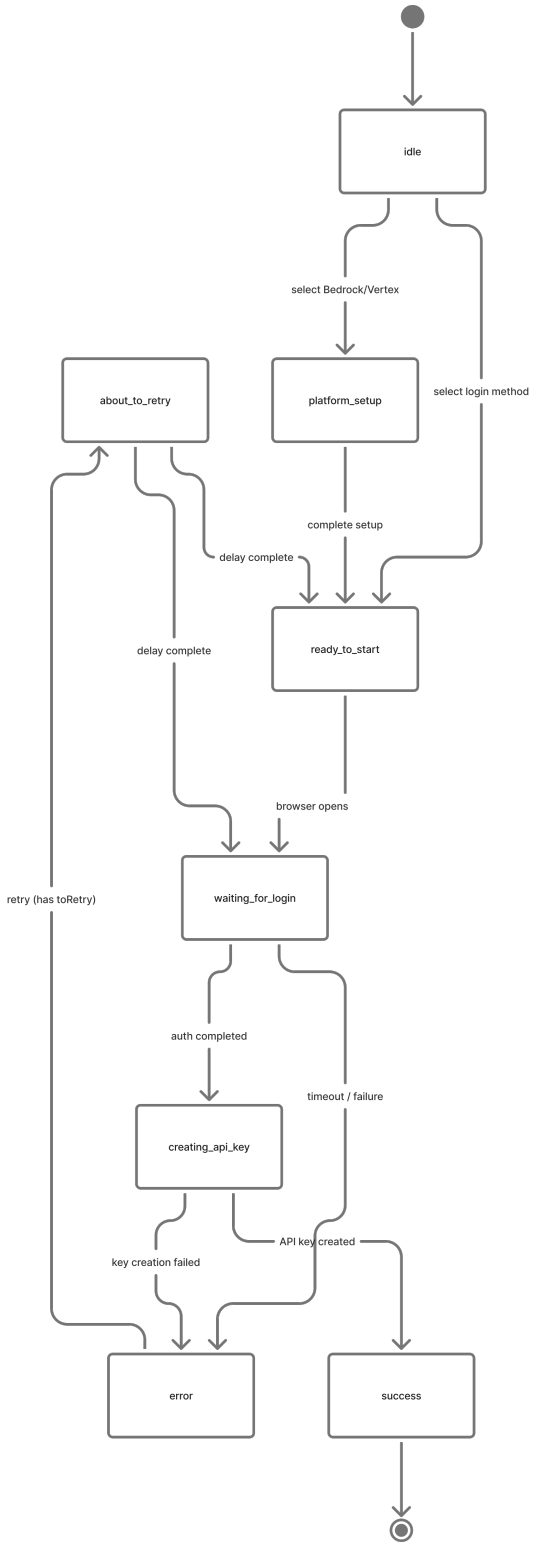
The flow starts at **idle** where the user picks a login method. Selecting claude.ai or console goes to **ready_to_start**, while selecting a provider that needs platform-specific setup goes through **platform_setup** first. From **ready_to_start**, a browser opens for login (**waiting_for_login**). After browser auth, the flow creates an API key (**creating_api_key**). Success lands at the **success** terminal state. Any error goes to **error**, which may auto-retry via the **about_to_retry** state.

Property	Value
Source	src/components/ConsoleOAuthFlow.tsx:26–52
Implementation	Explicit (union type + React useState)
States	idle, platform_setup, ready_to_start, waiting_for_login, creating_api_key, about_to_retry, success, error
Verification	VALID (0 errors)

Design Notes

The **about_to_retry** state is a clever polymorphic retry mechanism—it stores which state to return to after the delay, allowing a single retry pattern to serve multiple error sources. The OAuthStatus union type has 8 variants, each carrying context-specific data.

Figure 3: OAuth Authentication Flow State Diagram



4. Voice Input

Claude Code supports push-to-talk voice input. The user holds a key to record, releases to stop, and the audio is transcribed via speech-to-text.

How It Works

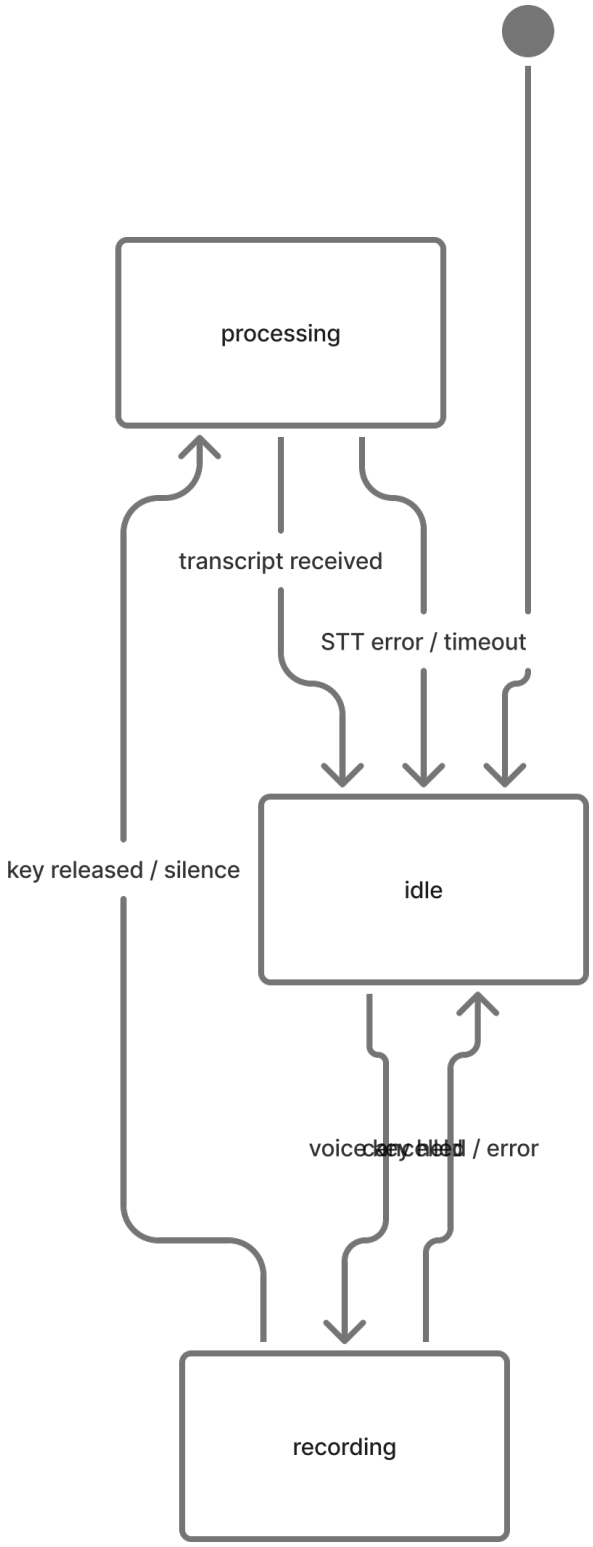
Pressing the voice key transitions from **idle** to **recording**, which activates the microphone via the native audio module. Releasing the key (or hitting a silence timeout) moves to **processing**, where audio is sent for transcription. When the transcript arrives (or an error/timeout occurs), the machine returns to **idle**.

Property	Value
Source	src/hooks/useVoice.ts
Implementation	Implicit (React useState + timers + generation counters)
States	idle, recording, processing
Verification	VALID (0 errors)

Design Notes

The `sessionGenRef` counter is the key design insight. Each recording session gets a generation number. When a callback fires, it checks whether its generation matches the current one—if not, the callback is from a stale session and is silently discarded. This prevents zombie connections from corrupting state. A 200ms release timeout smooths key-up detection.

Figure 4: Voice Input State Diagram



5. Feedback Survey

After certain interactions, Claude Code can prompt users for feedback. This state machine manages the full feedback collection lifecycle, from trigger through optional transcript sharing to completion.

How It Works

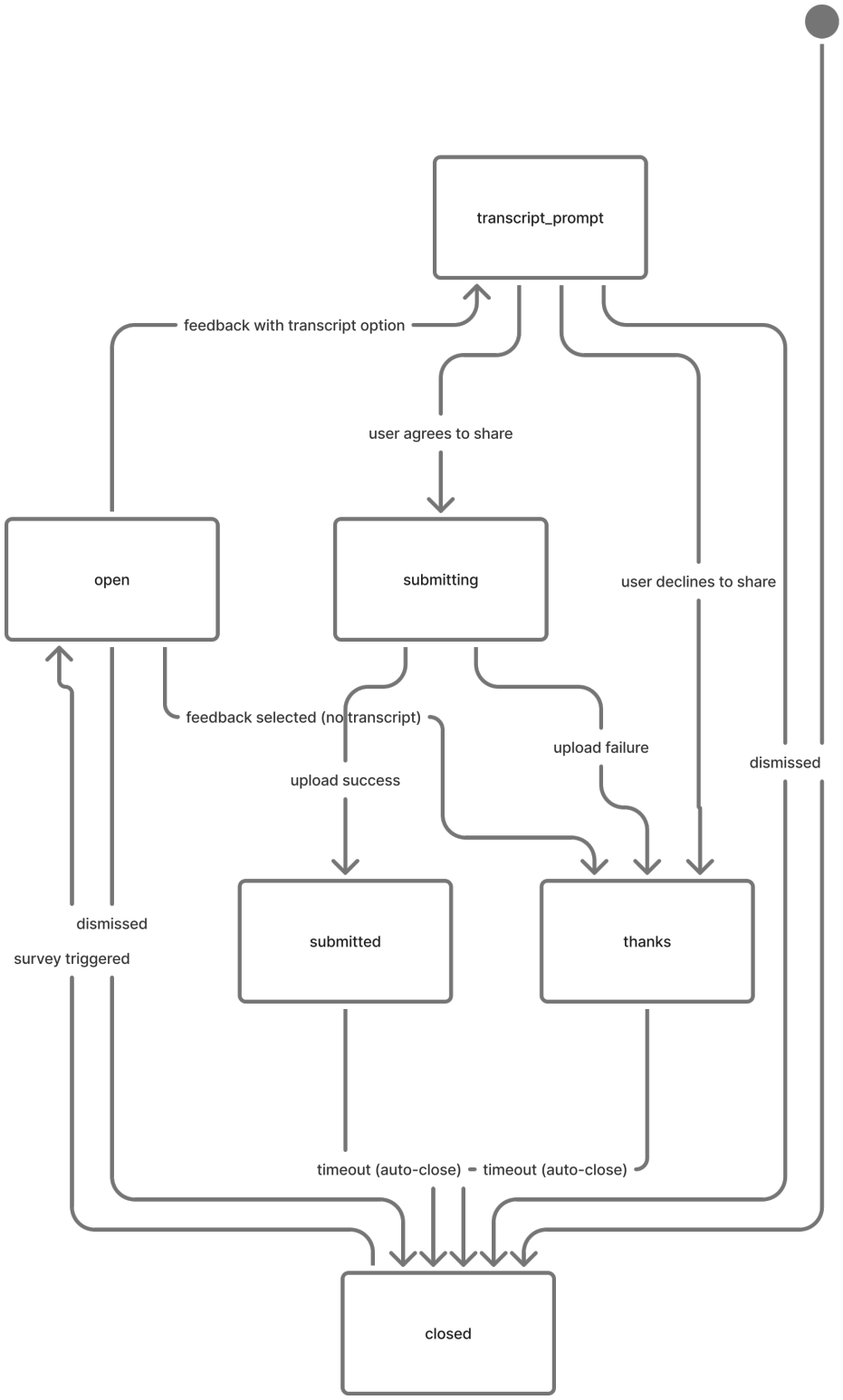
The survey starts **closed** and opens on a trigger event. The user selects a feedback option. If the option supports transcript sharing, the flow goes through **transcript_prompt** → **submitting** → **submitted**. Simple feedback goes directly to **thanks**. Both terminal display states auto-close after a timeout.

Property	Value
Source	src/components/FeedbackSurvey/useSurveyState.tsx
Implementation	Implicit (React useState + callbacks)
States	closed, open, transcript_prompt, submitting, thanks, submitted
Verification	VALID (0 errors)

Design Notes

The dual terminal states (**thanks** vs **submitted**) distinguish simple feedback from transcript-sharing feedback in the UI. `showSubmittedThenClose()` is called on successful transcript upload, producing the **submitted** state. `showThanksThenClose()` handles the simpler path.

Figure 5: Feedback Survey State Diagram



6. Worktree Exit Dialog

When an agent session running in a git worktree ends, this dialog determines what to do with the worktree. If there are unsaved changes, the user gets a choice; if it's clean, automatic cleanup begins.

How It Works

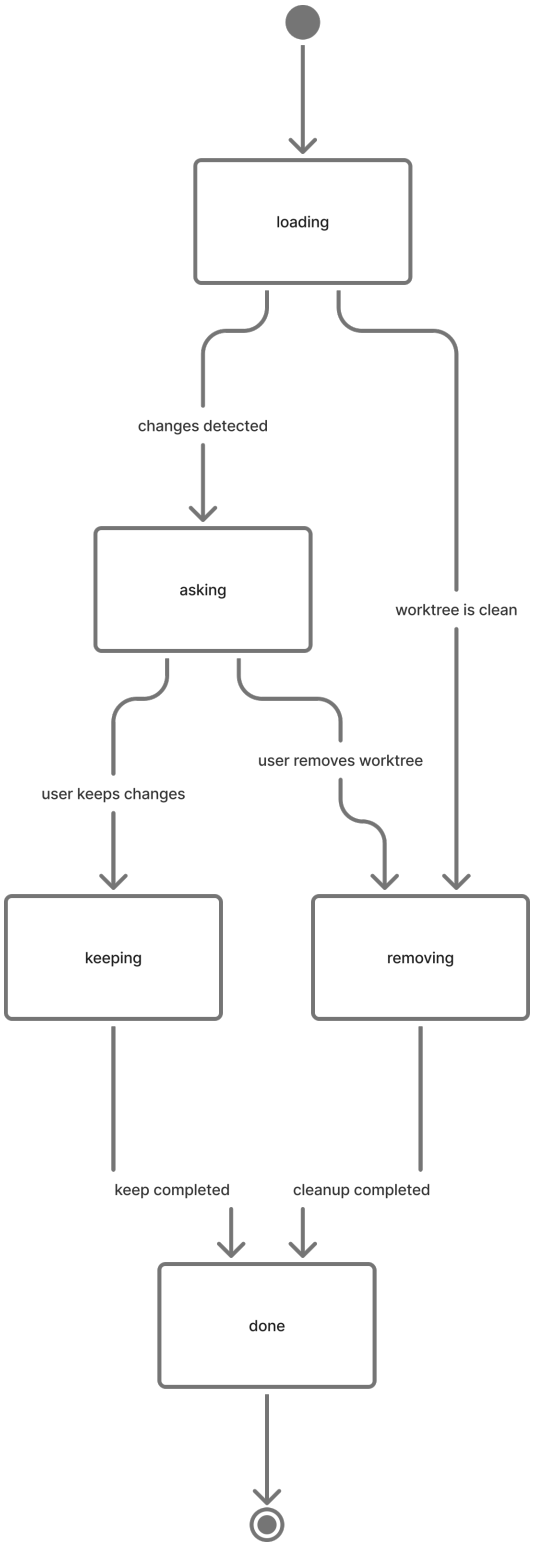
On mount, the dialog enters **loading** and checks git status and commit history against the original branch. If the worktree is clean, it auto-transitions to **removing**. If changes are found, it goes to **asking** where the user can **keep** or **remove**. Both paths end at **done**.

Property	Value
Source	src/components/WorktreeExitDialog.tsx
Implementation	Implicit (React useState + guards)
States	loading, asking, keeping, removing, done
Verification	VALID (1 warning, 0 errors)

Design Notes

The **loading** state acts as an automatic decision point with two guarded transitions. The guards (`worktree_is_clean` and `worktree_has_changes`) are logically complementary but Orca cannot statically prove this, resulting in a guard exhaustiveness warning.

Figure 6: Worktree Exit Dialog State Diagram



7. Vim Normal Command Parser

Claude Code includes a vim mode for its text input. This state machine implements the full vim grammar for NORMAL mode commands—the most complex machine in the codebase with 12 states and 51 transitions.

How It Works

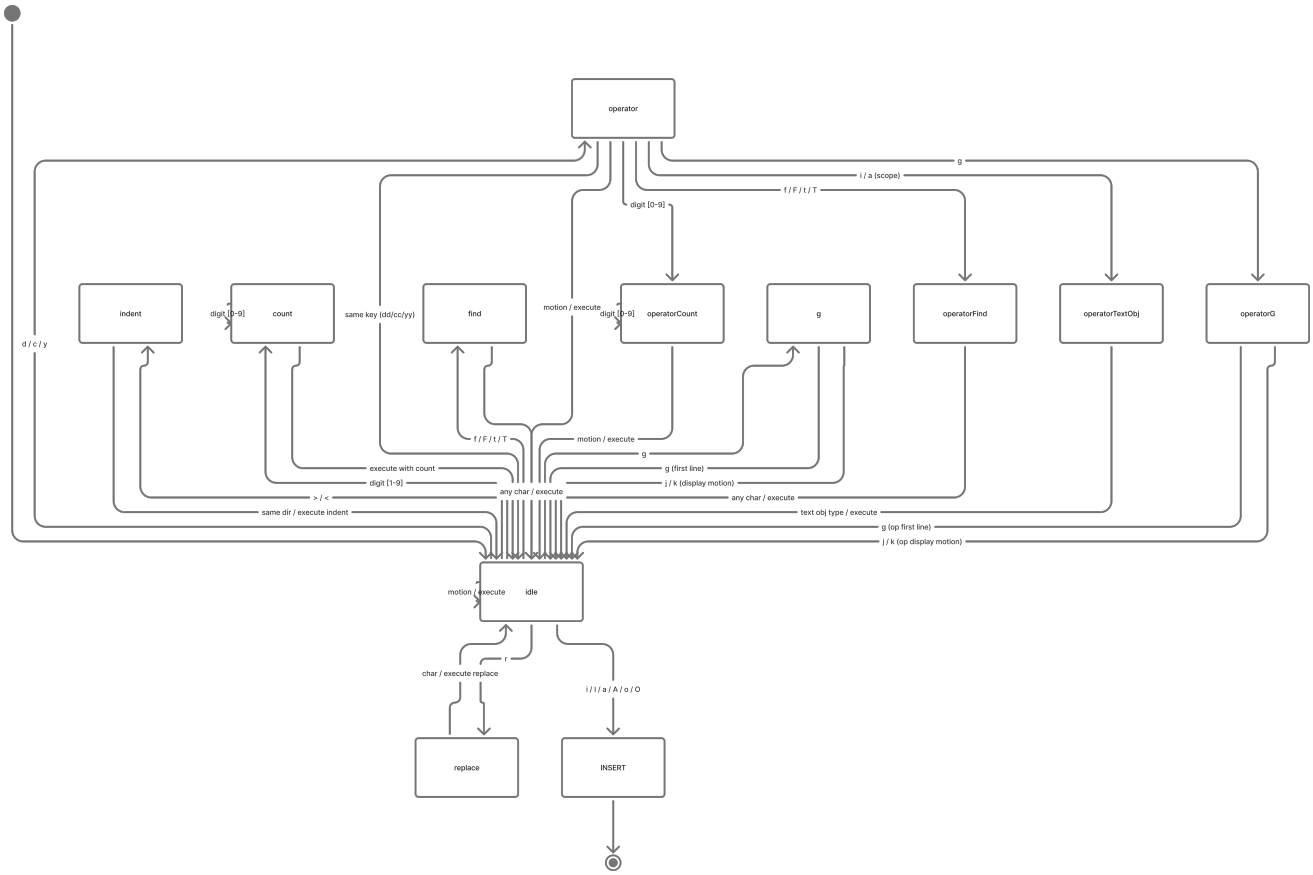
The parser starts at **idle**. A digit key starts a **count** prefix. An operator key (d/c/y) enters the **operator** state waiting for a motion or text object. Find keys (f/F/t/T) enter **find** waiting for a character. The **g** prefix has its own state for gj/gk/gg motions. Operators compose: **operator** + find key → **operatorFind**, **operator** + text object scope → **operatorTextObj**. Every state ultimately resolves back to **idle** after execution.

Property	Value
Source	src/vim/types.ts:59–75, src/vim/transitions.ts:59–88
Implementation	Explicit (discriminated union + pure transition function)
States	idle, count, operator, operatorCount, operatorFind, operatorTextObj, find, g, operatorG, replace, indent, insert
Verification	VALID (0 errors)

Design Notes

This is a textbook state machine implementation with a pure `transition()` function that dispatches to per-state handlers. Each state knows exactly what input it's waiting for, and TypeScript's exhaustive switch ensures no state is missed. The `CommandState` discriminated union has 11 variants, making it the richest type-level state machine in the codebase.

Figure 7: Vim Normal Command Parser State Diagram



Cross-Cutting Patterns

Implementation Patterns

- **Explicit state machines** (Vim, MCP, OAuth): Use TypeScript discriminated unions where each state variant is a distinct type. Transitions are pure functions. These are the most verifiable.
- **Implicit state machines** (Session, Voice, Feedback, Worktree): Use React useState with string literal types. Transitions are scattered across event handlers and effect callbacks. Orca modeling reveals their hidden structure.
- **Guard patterns:** The MCP connection uses `can_retry` guards on retry transitions. The Worktree dialog uses complementary guards (`worktree_is_clean` / `worktree_has_changes`) for automatic decisions.
- **Concurrency control:** The Voice machine uses generation counters to invalidate stale callbacks—a pattern that implements implicit event cancellation without formal cancel events.

Verification Summary

All 7 machines pass Orca verification. Six pass cleanly; the Worktree Exit Dialog has one guard exhaustiveness warning that is a false positive (the guards are logically complementary but Orca cannot prove it statically).

Recommendations

- The Vim command parser already follows pure-function state machine architecture. Consider adopting Orca-generated XState code for the simpler machines to get formal guarantees.
- The MCP connection lifecycle would benefit from a formal state machine library—its 5-state, 9-event model with retry guards is complex enough to warrant explicit modeling.
- Add a default/else transition in the Worktree Exit Dialog for the status_checked event to satisfy static guard exhaustiveness.
- Document the Voice Input generation-counter pattern as a formal concurrency strategy, since it implements implicit event cancellation.
- Consider extracting Orca .orca.md files alongside the source code for the explicit machines (Vim, MCP, OAuth) as living documentation.

Appendix A: Detailed Orca Analysis

This appendix contains the full Orca model details for each state machine, including states, events, transition tables, and verification results.

A.1. Session Lifecycle

Source: `src/utils/sessionState.ts`

Metrics: 3 states | 5 events | 5 transitions

Orca Model

States: `idle [initial], running, requires_action`

Events: `submit, processing_complete, permission_required, action_provided, action_denied`

Transition Table

Transition
idle+submit→running
running+processing_complete→idle
running+permission_required→requires_action
requires_action+action_provided→running
requires_action+action_denied→idle

Verification Result

Status: VALID Errors: 0

Clean pass. All events are properly scoped to their valid states via ignore directives.

A.2. MCP Server Connection

Source: `src/services/mcp/types.ts:180–226`

Metrics: 5 states | 9 events | 13 transitions

Orca Model

States: `pending [initial], connected, needs_auth, failed, disabled`

Events: `connect_success, auth_required, connection_error, runtime_error, reconnect, disable, provide_auth, retry, enable`

Transition Table

Transition
pending+connect_success→connected
pending+auth_required→needs_auth
pending+connection_error→failed [guard: can_retry]
pending+disable→disabled
connected+runtime_error→failed
connected+reconnect→pending
connected+disable→disabled
needs_auth+provide_auth→pending
needs_auth+disable→disabled
failed+retry→pending [guard: can_retry]
failed+disable→disabled
disabled+enable→pending

Verification Result

Status: VALID Errors: 0

Clean pass. The can_retry guard on retry and connection_error transitions correctly gates reconnection attempts.

A.3. OAuth Authentication Flow

Source: `src/components/ConsoleOAuthFlow.tsx:26–52`

Metrics: 8 states | 12 events | 18 transitions

Orca Model

States: `idle [initial]`, `platform_setup`, `ready_to_start`, `waiting_for_login`, `creating_api_key`, `about_to_retry`, `success [final]`, `error`

Events: `select_login_method`, `select_platform`, `complete_platform_setup`, `browser_opened`, `browser_auth_completed`, `api_key_created`, `api_key_failed`, `auth_timed_out`, `retry`, `delay_complete`, `unexpected_error`, `dismiss`

Transition Table

Transition
idle+select_login_method→ready_to_start
idle+select_platform→platform_setup

Transition
platform_setup+complete_platform_setup→ready_to_start
ready_to_start+browser_opened→waiting_for_login
waiting_for_login+browser_auth_completed→creating_api_key
waiting_for_login+auth_timed_out→error
creating_api_key+api_key_created→success
creating_api_key+api_key_failed→error
error+retry→about_to_retry [guard: has_to_retry]
about_to_retry+delay_complete→ready_to_start
any+unexpected_error→error

Verification Result

Status: VALID Errors: 0

Clean pass. All 12 events properly handled or ignored in every state.

A.4. Voice Input

Source: `src/hooks/useVoice.ts`

Metrics: 3 states | 7 events | 7 transitions

Orca Model

States: `idle [initial], recording, processing`

Events: `voice_key_down, voice_key_up, silence_timeout, cancel, transcript_received, stt_error, stt_timeout`

Transition Table

Transition
idle+voice_key_down→recording
recording+voice_key_up→processing
recording+silence_timeout→processing
recording+cancel→idle
processing+transcript_received→idle
processing+stt_error→idle
processing+stt_timeout→idle

Verification Result

Status: VALID Errors: 0

Clean pass. Events like voice_key_up and stt_error are correctly ignored in states where they cannot logically occur.

A.5. Feedback Survey

Source: `src/components/FeedbackSurvey/useSurveyState.tsx`

Metrics: 6 states | 9 events | 11 transitions

Orca Model

States: `closed [initial], open, transcript_prompt, submitting, thanks, submitted`

Events: `trigger, select_feedback, select_feedback_with_transcript, dismiss, decide_transcript_yes, decide_transcript_no, upload_success, upload_failure, timeout`

Transition Table

Transition
closed+trigger→open
open+select_feedback_with_transcript→transcript_prompt
open+select_feedback→thanks
open+dismiss→closed
transcript_prompt+decide_transcript_yes→submitting
transcript_prompt+decide_transcript_no→thanks
transcript_prompt+dismiss→closed
submitting+upload_success→submitted
submitting+upload_failure→thanks
thanks+timeout→closed
submitted+timeout→closed

Verification Result

Status: VALID Errors: 0

Clean pass. The submitted state is correctly reachable via the submitting+upload_success transition.

A.6. Worktree Exit Dialog

Source: `src/components/WorktreeExitDialog.tsx`

Metrics: 5 states | 5 events | 6 transitions

Orca Model

States: `loading [initial], asking, keeping, removing, done [final]`

Events: `status_checked, keep_changes, remove_worktree, keep_completed, cleanup_completed`

Transition Table

Transition
loading+status_checked→removing [guard: worktree_is_clean]
loading+status_checked→asking [guard: worktree_has_changes]
asking+keep_changes→keeping
asking+remove_worktree→removing
keeping+keep_completed→done
removing+cleanup_completed→done

Verification Result

Status: VALID (1 warning) Errors: 0

One GUARD_EXHAUSTIVENESS warning: Orca cannot statically prove that worktree_is_clean and worktree_has_changes are logically complementary. In practice they always are, since the git status check produces exactly one of these conditions.

A.7. Vim Normal Command Parser

Source: `src/vim/types.ts:59–75, src/vim/transitions.ts:59–88`

Metrics: 12 states | 2 events | 51 transitions

Orca Model

States: `idle [initial], count, operator, operatorCount, operatorFind, operatorTextObj, find, g, operatorG, replace, indent, insert`

Events: `key, empty_input`

Transition Table

Transition
idle+key→count [isDigit1to9]
idle+key→operator [isOperatorKey]
idle+key→find [isFindKey]

Transition
idle+key→g [isGKey]
idle+key→replace [isReplaceKey]
idle+key→indent [isIndentKey]
idle+key→idle [isSimpleMotion]
idle+key→insert [isInsertCommand]
operator+key→idle [isSameOperator—line op]
operator+key→operatorCount [isDigit0to9]
operator+key→operatorTextObj [isTextObjScope]
operator+key→operatorFind [isFindKey]
operator+key→operatorG [isGKey]
operatorFind+key→idle
find+key→idle
replace+empty_input→idle
replace+key→idle
indent+key→idle [isIndentSameDir]
… +33 more (see source for full table)

Verification Result

Status: VALID Errors: 0

Clean pass with all 51 transitions verified. All states are reachable and no deadlocks exist. The empty_input event is correctly scoped to the replace state only.

Appendix B: Orca Verification Error Codes

Reference for error codes produced by Orca's verify_machine tool:

Error Code	Description
UNREACHABLE_STATE	A state cannot be reached from the initial state via any transition path. Indicates dead code or a missing transition.
INCOMPLETE_EVENT_HANDLING	A state does not explicitly handle an event. Requires either a transition or an ignore directive.
GUARD_EXHAUSTIVENESS	Multiple guarded transitions for the same state+event may not cover all possible context values.
NO_INITIAL_STATE	No state is marked with [initial]. Exactly one initial state is required.
DEADLOCK	A non-final state has no outgoing transitions. The machine can enter but never leave this state.
GUARD_DETERMINISM	Multiple transitions for the same state+event have overlapping guard conditions, creating ambiguity.